

Tutor: Sun Pengzhan
pengzhan@u.nus.edu

Theoretical:

Histogram Equalization (Lecture 2)

Q1: Consider images of size 12x12 with pixel values ranging from 0 to 15. For the following image histograms, estimate the mapped intensity values after histogram equalization.

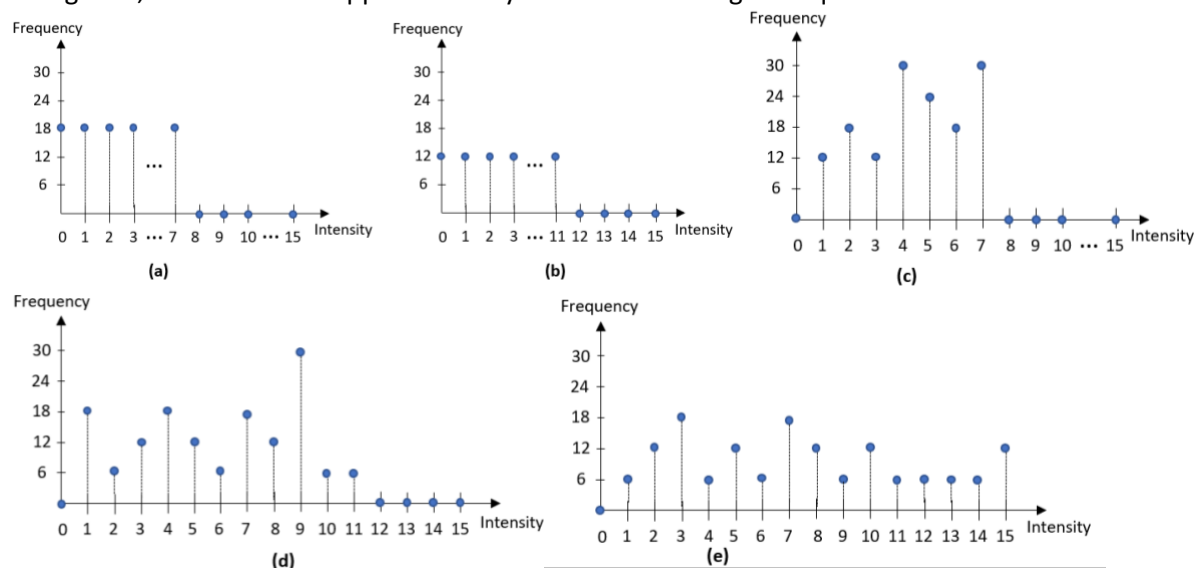


Figure 1

Original Intensity	Intensity after histogram equalization				
	(a)	(b)	(c)	(d)	(e)
0					
1					
2					
3					
4					
5					
6					
7					
8					
9					
10					
11					
12					
13					
14					
15					

Filtering (Lecture 2)

Q2: Consider the source image shown in (a) and the filter kernel shown in (b). What is the resulting value at positions (0,0), (2,2) and (4,3) if you were to apply cross-correlation, convolution, and 3x3 median filtering, with the use of zero-padding on the borders? Positions are denoted as (row,column) in the unpadded image.

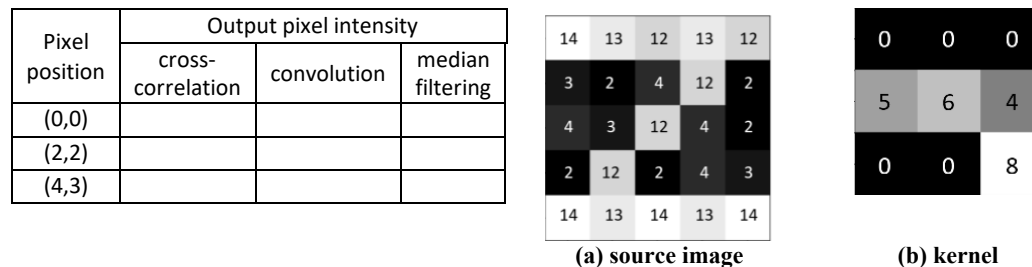


Figure 2

Coding (see CS4243-Tutorial-01.ipynb):

Basic Operations on Images (Lecture 1)

- Install basic python packages: `imutils`, `opencv-python`, `scikit-image`
- Load and display `House.jpg`
- Visualize the (R,G,B) channels; visualize the normalized colour and intensity images
- Convert RGB image into HSV colour space and visualize the (H,S,V) channels
- Resize an image such that the dimensions are 2X, 0.5X and 0.05X the original size

Operations on Images (Lecture 2)

- For `Capture.jpeg`
 - Visualize the image histogram
 - Access a pixel value: what is the pixel intensity of (100,800)
 - Access a window of data: crop out the stadium
- Write your own algorithm for histogram stretching and equalization. Visualize the resulting histograms and compare with the built-in `cv2.equalizeHist`¹.

Image Filtering (Lecture 2)

- Smooth image `Synthetic.png` with a box filter of size 5 and a Gaussian kernel of size 5 and a sigma of size 3.
- Smooth image `Building.png` with a box filter of size 30 and a Gaussian kernel of size 30 and a sigma of size 10.
- Apply 2D Laplace filter as described in L3 slide 25 to images `Synthetic.jpg` and `Sharpen.jpeg` and the smoothed versions from above

Past Quiz Questions:

- AY2021_Quiz1
- AY2122_Quiz1: Q1 to Q4

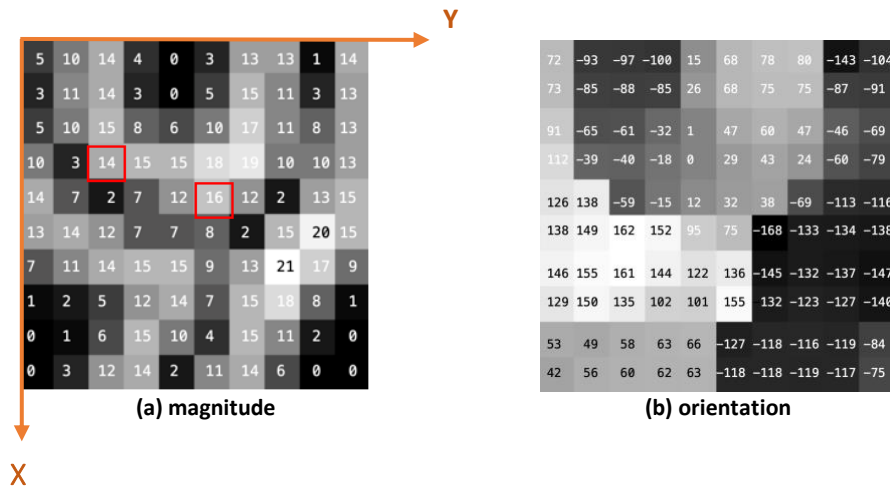
¹ https://docs.opencv.org/4.x/d5/daf/tutorial_py_histogram_equalization.html

Tutor: Sun Pengzhan
pengzhan@u.nus.edu

Theoretical:

Canny Edge Non-Maximum Suppression (Lecture 3)

Q1: For the following magnitude and orientation images, estimate the edge image value after non-maximum suppression for the pixels located at (3,2) and (4,5).



Coding (see CS4243-Tutorial-02.ipynb):

Canny Edge Detection (Lecture 3)

- Add Gaussian noise with $\mu=0$ and $\sigma=20$ to the image `Nemo.jpeg` to produce a noisy image. After adding Gaussian noise, apply a floor and ceiling operation to ensure that all pixels remain within [0-255].
- Apply the `cv2.Sobel` with `size=3` to the noisy outputs and visualize the outputs
- Smooth the noisy image with a Gaussian filter of `size=15` and `sigma={1, 3, 9}` and compare
- Apply the `cv2.Sobel` with `size=3` to the different smoothed images and compare the outputs. Are there any differences in the extracted edges?
- Apply built-in `cv2.Canny` to the image `Pole.png`, using `{50, 100}` as `{threshold1, threshold2}`. Compare these outputs when `{80, 100}` and `{50, 300}` as thresholds. Are there any differences in the extracted edges?

Past Quiz Questions:

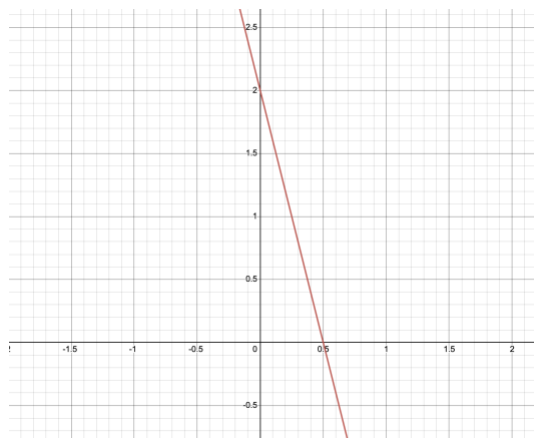
- AY2021_Quiz2: Q1
- AY2122_Quiz1: Q5

Tutor: Sun Pengzhan
pengzhan@u.nus.edu

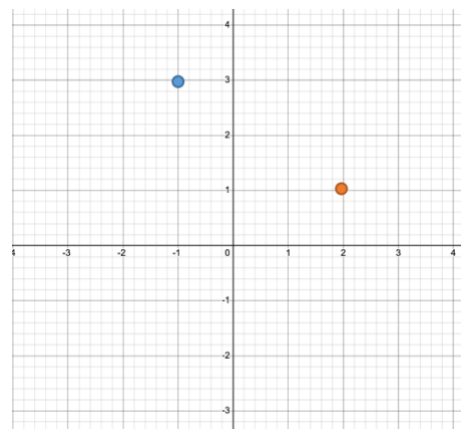
Theoretical:

Q1: Consider the plot of a straight line in (a). Give the equation of the line in slope-intercept, double-intercept and normal form.

Q2: Consider the two points $p_1 = (2,1)$ and $p_2 = (-1,3)$ shown in (b). Using the slope-intercept form of a line ($y = mx + b$), estimate the Hough accumulator values for detecting a straight line. For the accumulator $A(m, b)$, use a range of $(-1 \leq m < 5, -1 \leq b < 5)$, where m and b increment by values of 1. What is the detected line based on the maximum found in the accumulator and how does this compare to the true line?



(a)



(b)

Coding:

- Apply `cv2.HoughLines` to `Line.jpg`, using 80 and 50 as thresholds and compare the results
- Apply `cv2.HoughCircles` to `Circle.jpg`, and investigate the influence of param1 and param2
 - Blur the image with Median filter with size=5
 - Set circle radius with minRadius=50, maxRadius=65 and minDist=40
 - Try to set param1 and param2 as {80, 20}, {80, 10} and {50, 20}, and see the difference on outputs
 - Apply circle detection on the unblurred image using the best parameters found above and check for differences in the output

Past Quiz Questions:

- AY2122_Quiz1: Q6